



SEEK WISDOM, ELEVATE YOUR INTELLECT AND SERVE HUMANITY !



Addis Ababa University

Addis Ababa institute of Technology

School of Information Technology and Engineering

Section 1

Course: SE 4801 — Enterprise Application Development

Assignments 1

Name

ID No

Hailemichale Lijalem

(ATE/1051/14)

School of information Technology and Engineering

Addis Ababa Institute of Technology

Mr. Abel

Apr 3, 2026

Section A

A1. Enterprise Applications & Spring Boot

Question A1

Describe three characteristics that distinguish an enterprise application from a standard desktop application. For each characteristic, give a concrete example of how Spring Boot addresses it.

Answer:

1. High Concurrency & Scalability

- Enterprise applications must handle millions of concurrent users and traffic spikes. But a desktop application usually serves one user.
- How Spring Boot addresses this: Spring Boot supports horizontal scaling, embedded servers (Tomcat), and integrates with cloud environments.

2. Strict Non-Functional Requirements (NFRs)

- Enterprise apps must meet strict requirements:
 - High performance
 - Reliability
 - Security
 - Observability
- But for Desktop apps do not require these production-level guarantees.
- How Spring Boot addresses this:
 - Spring Security for Authentication/authorization
 - Spring Actuator for health checks and monitoring
 - Built-in performance optimizations via auto-configuration

3. Maintainability & Long-Term Evolution

- Enterprise systems are maintained by large teams (500+ engineers). Code must follow SOLID principles and layered architecture.
- Desktop apps usually have smaller scope and shorter lifecycle.
- How Spring Boot addresses this:
 - Promotes Layered Architecture (Controller → Service → Repository)
 - Encourages solid principles
 - Uses Dependency Injection

A2. Monolith vs Microservices

Question A2

A start-up is building a food delivery platform. The team has 3 developers and a 3-month deadline. (a) Recommend an appropriate architecture (monolith or microservices) and justify your choice with two specific reasons. (b) Describe the Strangler Fig pattern and explain when it would be appropriate to migrate from the initial architecture to microservices.

Answer:

(a) Recommended Architecture: Monolith

For a startup with:

- 3 developers
- 3-month deadline

A monolithic architecture is recommended.

Two reasons:

- Easier to develop and debug early on
Unit 1 states monoliths are simpler, have one codebase, and no distributed systems complexity.
- Faster to start
No need for DevOps maturity, service discovery, or network communication overhead

(b) Strangler Fig Pattern

- The Strangler Fig pattern means gradually replacing parts of a monolithic system with microservices over time.
 - Instead of rewriting everything:
 - Extract one component at a time
 - Route traffic to the new service
 - Slowly remove the old functionality
- When to migrate to microservices:
 - migrate only when:

- A component needs independent scaling
- Deploy frequency differs between teams
- Technology stacks must differ
- Team size exceeds the “two-pizza rule”

A3. SOLID Principles

Question A3

The following class violates TWO of the SOLID principles. Identify both violations by name, explain why each is a violation, and briefly describe how you would refactor the class to fix each one.

Answer:

➤ **Single Responsibility Principle (SRP)**

Violation:

The class performs:

- **Validation**
- **Payment processing**
- **Email sending**
- **Database operations**

It has multiple reasons to change, which violates SRP.

Split into:

- **Order Validator**
- **Payment Service**
- **Email Service**
- **Order Repository**

Each class should have one responsibility.

➤ **Dependency Inversion Principle (DIP)**

Violation:

The class directly creates:

- **HttpClient**
- **SMTP configuration**
- **Database connection**

It depends on concrete implementations, not abstractions.

Refactor:

Inject dependencies using interfaces:

```
“class OrderProcessor {  
  
    private final PaymentGateway payment;  
  
    private final NotificationService email;  
  
    private final OrderRepository repo;  
  
” so if we use injection so it depends on abstraction
```

A4. Maven Build Lifecycle

Question A4

- List the six core Maven lifecycle phases in the correct order, starting from validate.
- A developer runs `mvn package` and it fails with a compilation error. Which lifecycle phases ran before the failure?
- Explain the difference between `mvn package` and `mvn install` and when you would use each.

Answer:

- Six Core Maven Lifecycle phases in order

Phase	What It Does
1. validate	Checks that the project structure and pom.xml are valid before doing any work.
2. compile	Compiles the production source code in <code>src/main/java</code> .
3. test	Compiles and runs unit tests in <code>src/test/java</code> using JUnit.
4. package	Bundles the compiled code into a JAR or WAR file in the <code>target/</code> directory.
5. verify	Runs integration tests and checks to confirm the package is valid.
6. install	Copies the artifact into the local Maven repository (.m2) for other local projects to use.

- (b) Phases That Ran Before a Compilation Failure
 - If mvn package fails at compilation:
 - The phases that ran before failure:
 - validate
 - compile
 - It fails during compile, so test and package do not complete.
- (c) difference between mvn package and mvn install
 - ✓ **mvn package**
 - Compiles
 - Runs tests
 - Creates executable JAR/WAR
 - Used for building deployable artifact
 - ✓ **mvn install**
 - Does everything package does
 - Copies artifact to local Maven repository

So we use install when another local project depends on your artifact.

A5. Spring IoC & Dependency Injection

Question A5

- (a) Explain what Inversion of Control (IoC) means and how Spring's container implements it.
- (b) Compare constructor injection vs field injection in Spring. Which is preferred and why?
- (c) What problem does dependency injection solve when it comes to unit testing? Explain with a brief example.

Answer:

- (a) what Inversion of Control (IoC)

- Inversion of Control means the framework controls object creation instead of the class creating its own dependencies.

With Spring IoC:

- The container creates and injects dependencies

(b) Compare constructor injection vs field injection in Spring

Constructor Injection	Field Injection
• Dependencies provided through constructor	• Uses <code>@Autowired</code> on fields
• Immutable	• Uses <code>@Autowired</code> on fields
• Preferred method	• Not recommended

So constructor injection is best practice.

(c) What problem does dependency injection solve when it comes to unit testing?

- Problem DI solves in unit testing
- Without DI:
 - Cannot replace dependencies
 - Hard to mock external services
- With DI:
 - we can inject mock objects

Example:

Inject mock `PaymentService` into `OrderService` during testing.

A6. Layers of a Spring Boot Application

Question A6

Draw and label the four standard layers of a Spring Boot application (Controller → Service → Repository → Database).

For each layer: (a) state its single responsibility, (b) name the Spring annotation(s) used to declare a class in that layer, (c) explain what should not live in that layer.

Answer:

Layer Diagram

Controller

↓

Service

↓

Repository



Database

Layer	Annotation	Single Responsibility	Should NOT Contain
Controller	@RestController, @RequestMapping, @GetMapping etc.	Accepts HTTP requests, validates input with @Valid, maps request/response DTOs, returns the correct HTTP status code.	Business logic, database queries, or domain rules. The controller should be thin; it delegates immediately to the service.
Service	@Service, @Transactional	Contains all business logic, calculations, domain rules, transaction boundaries, and orchestration of multiple repository calls.	SQL queries, direct JPA calls, or any HTTP-specific objects.
Repository	@Repository, extends JpaRepository	Data access only; translates between Java entity objects and the database. Provides CRUD and custom query methods.	Business logic, input validation, or anything HTTP-related. Repositories deal only with persistence.
Database	(application.properties)	Reliable storage and retrieval of persistent data.	Application logic of any kind. All logic lives in the layers above.

Section B

B1. Java Collections & Generics

Question B1.1

write what the output will be. If a line throws an exception, write the exception name and explain why?

Answer:

Line	Expected Output/ Exception	Explanation
1	"apple"	Collections.sort() sorts strings lexicographically. The list becomes ["apple","banana","cherry"]. list.get(0) returns "apple".
2	2	Maps do not allow duplicate keys. The second "Alice" replaces the first.. The map has exactly two entries. size() returns 2
3	95	Latest value for Alice is 95 .
4	3	Sets do not allow duplicates
5	ClassCastException	items.get(1) retrieves the Integer 42. The cast (String) fails at runtime because Integer is not a subtype of String.

Question B1.2

Complete the following generic method find Max. It must work for any type that implements Comparable. Then write two test cases demonstrating it with (a) a list of Integers, (b) a list of Strings.

Answer:

The bounded type parameter `>` means T can be any type that implements Comparable, which guarantees `compareTo()` is available. We iterate keeping track of the largest element seen so far.

```

if (list == null || list.isEmpty()) {
    throw new IllegalArgumentException("list must not be null or empty");
}
T max = list.get(0);
for (T element : list) {
    if (element.compareTo(max) > 0) {
        max = element;
    }
}
return max;
}

// Test (a): Integers
List ints = List.of(3, 1, 7, 2, 9, 4);
System.out.println(findMax(ints)); // Output: 9

// Test (b): Strings — compared lexicographically
List words = List.of("banana", "apple", "cherry", "mango");
System.out.println(findMax(words)); // Output: mango

```

B2. Object-Oriented Programming

Question B2.1

Design a small class hierarchy for a ShopWave product catalogue. Requirements:

- (a) Create an abstract class `Product` with fields: `id`, `name`, `price`, `stock`.
- (b) Add a method `calculateDiscount(double percentage)` — abstract.
- (c) Create two concrete subclasses: `PhysicalProduct` (extra field: `weightKg`) and `DigitalProduct` (extra field: `downloadUrl`).
- (d) Implement `calculateDiscount` differently in each subclass.
- (e) Override `toString()` in both subclasses to include all relevant fields.

Answer:

- The hierarchy uses an abstract class as the base because products share common fields but must calculate discounts differently. Declaring `calculateDiscount` as abstract forces every subclass to provide its own implementation; the compiler will refuse to compile a subclass that does not.

```

abstract class Product {
    protected Long id;
    protected String name;
    protected double price;
    protected int stock;

    public Product(Long id, String name, double price, int stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    public abstract double calculateDiscount(double percentage);
}

class PhysicalProduct extends Product {
    private double weightKg;

    public PhysicalProduct(Long id, String name, double price, int stock, double weightKg) {
        super(id, name, price, stock);
        this.weightKg = weightKg;
    }

    @Override
    public double calculateDiscount(double percentage) {
        return price * (1.0 - (percentage / 100.0));
    }

    @Override
    public String toString() {
        return "PhysicalProduct{id='" + id + "', name='" + name +
            "', price=" + price + ", stock=" + stock +
            ", weightKg=" + weightKg + "'}";
    }
}

class DigitalProduct extends Product {
    private String downloadUrl;

    public DigitalProduct(Long id, String name, double price, int stock, String
downloadUrl) {
        super(id, name, price, stock);
        this.downloadUrl = downloadUrl;
    }
}

```

```

@Override
public double calculateDiscount(double percentage) {
    // Digital products get an extra 5% discount
    double effectivePercentage = Math.min(percentage + 5.0, 100.0);
    return price * (1.0 - (effectivePercentage / 100.0));
} @Override
public String toString() {
    return "DigitalProduct{id='" + id + "', name='" + name +
        "', price='" + price + "', stock='" + stock +
        "', downloadUrl='" + downloadUrl + "'}";
}
}

public class QuestionB2_1 {
    public static void main(String[] args) {
        PhysicalProduct pp = new PhysicalProduct(1L, "Book", 20.0, 50, 0.5);
        DigitalProduct dp = new DigitalProduct(2L, "E-Book", 15.0, 100, "url");

        System.out.println(pp);
        System.out.println("Discounted (10%): " + pp.calculateDiscount(10));

        System.out.println(dp);
        System.out.println("Discounted (10%): " + dp.calculateDiscount(10));
    }
}

```

Question B2.2

Explain the difference between method overloading and method overriding in Java. For each: (a) write a two-line code example, (b) state whether it is resolved at compile time or runtime, and (c) give one real-world scenario from a Spring Boot application where you would use it.

	Method Overloading	Method Overriding
Definition	Multiple methods in the SAME class share the same name but have different parameter lists.	A SUBCLASS provides its own implementation of a method already defined in its parent. The signature must be identical.
Code Example	<pre>void save(Product p) { ... } void save(Product p, boolean notify) { ... }</pre>	<pre>class Animal { void speak() {} } @Override class Dog extends Animal { void speak() {} }</pre>
Resolved At	Compile time; the compiler picks the correct overload based on argument types.	Runtime; the JVM uses dynamic dispatch to call the version belonging to the actual object type.
Spring Boot Scenario	ProductService could overload createProduct(): one version	A base EmailService defines sendWelcomeEmail(). A MockEmailService

	takes a full DTO, another takes individual parameters for a batch import.	overrides it to just log. Spring injects the right one based on <code>@Profile</code> .
--	---	---

B3. Java Streams & Functional Programming

Question B3.1

Rewrite each of the following for-loops as a single Stream pipeline. The behavior must be identical.

Answer:

The mapping is: a for-loop condition becomes. `filter()`, collecting into a list becomes. `collect()`, summing becomes. `mapToDouble().sum()`, building a map becomes `Collectors.toMap()`, and finding the maximum becomes `.max().orElseThrow()`.

Loop 1: collect names starting with 'A':

```
List<String> result1 = names.stream ()
    filter (name -> name.startsWith ("A"))
    collect (Collectors.toList ());
```

Loop 2: sum order totals over 100:

```
double total = orders.stream ()
    .filter (o -> o.getTotal () > 100)
    .mapToDouble( Order :: getTotal)
    .sum();
```

Loop 3: build a map of product name to price:

```
Map<String, Double> priceMap = products.stream ()
    .collect (Collectors.toMap ( Product :: getName, Product :: getPrice));
```

Loop 4: find the most expensive product or throw if empty:

```
Product maxPriceProduct = products.stream ()
    .max(Comparator.comparingDouble(Product :: getPrice))
    .orElseThrow(() -> new NoSuchElementException("No products found"));
```

Question B3.2

Study the following Stream pipeline and answer the questions.

Answer:

1. What does this pipeline produce? Describe the result in plain English.

- It produces:
A list of the 5 most recent shipped orders, sorted by creation date descending,
- formatted as: CustomerName – TotalAmount

2. Which operations are intermediate and which are terminal? List each operation and classify it.

Operation	Type	Why
<code>.filter(...)</code>	Intermediate	Removes non-SHIPPED elements and returns a new Stream. Nothing executes yet.
<code>.sorted(...)</code>	Intermediate	Reorders elements and returns a new Stream. Still no execution.
<code>.limit(5)</code>	Intermediate	Caps the stream at five elements and returns a new Stream.
<code>.map(...)</code>	Intermediate	Transforms each Order into a String and returns a Stream.
<code>.collect(...)</code>	Terminal	Triggers all the above operations and accumulates results into a List. Nothing ran before this line.

3. Why is this pipeline lazy? What does that mean for performance?

- Streams are lazy because intermediate operations execute only when a terminal operation runs.
 - This improves performance because operations are chained efficiently.
4. If the orders list has 1 000 000 elements and only 3 have status SHIPPED, how many Order objects does the sorted() operation receive? Explain
- Only 3 pass the filter
 - So sorted() receives only 3 elements.

B4. Exception Handling

Question B4.1

Explain the difference between checked exceptions, unchecked exceptions, and errors in Java. For each: (a) give the root class in the hierarchy, (b) give one concrete example, (c) state whether you must handle it or not and why

Answer:

Category	Root Class	Example	Must Handle
Checked Exception	java.lang.Exception (not RuntimeException)	IOException	Must handle or declare because compiler enforces it
Unchecked Exception	java.lang.RuntimeException	NullPointerException	No need to explicitly handle
Error	java.lang.Error	OutOfMemoryError	Should not be handled — serious JVM problem

B4.2: Three Bugs in readUserFile()

Bug 1: catch(Exception e) is too broad

Bug 2: the catch(FileNotFoundException e) block is unreachable

Bug 3: the finally block calls reader.close() without a null check

The fixed version:

```
public String readUserFile(String filename) {  
    try (BufferedReader br = new BufferedReader(new FileReader(filename))) {  
        return br.readLine();  
    } catch (FileNotFoundException e) {  
        throw new RuntimeException("File not found: " + filename);  
    } catch (IOException e) {  
        throw new RuntimeException("IO error occurred");  
    }  
}
```

B5. Java Concurrency & Modern Java 21

B5.1: Concurrency Concepts

1) Race Condition

Occurs when multiple threads modify shared data simultaneously

```
synchronized void increment() {  
    count++;  
}
```

2) sleep() vs wait()

- Thread.sleep():
Pauses thread for fixed time.
- Object.wait():
Releases lock and waits for notification.
- Use sleep for delays.
Use wait for inter-thread coordination.

3) Volatile

Ensures visibility of changes across threads

Example in Spring Boot:

Shutdown flag in background task.

4) ExecutorService Example

```
ExecutorService executor = Executors.newFixedThreadPool(3);  
executor.submit(() -> task1());  
executor.submit(() -> task2());  
executor.submit(() -> task3());  
executor.shutdown();  
executor.awaitTermination(1, TimeUnit.MINUTES);
```

B5.2 Records & Virtual Threads

Record Version

```
public record ProductDTO(  
    Long id,  
    String name,  
    double price,  
    String category  
) {}
```


Records:

- Immutable data carriers with automatic constructor, getters, equals, hashCode.

Virtual Threads:

- Lightweight threads introduced in Java 21

Difference:

- Platform threads → OS-managed
- Virtual threads → JVM-managed, lightweight

Why important for Spring Boot:

- Handle thousands of concurrent requests efficiently

B5.3 Sealed Classes

- **How the code works:** The sealed interface Shape explicitly restricts its implementations to just three types: Circle, Rectangle, and Triangle. These permitted types are defined as records, meaning they are concise and unchangeable data structures. Inside the describe() method, a pattern-matching switch expression evaluates the object's actual type at runtime while automatically binding it to a newly typed variable, granting immediate access to the record's fields.
- **The impact of removing sealed:** If the sealed modifier is deleted, Shape reverts to a standard, unrestricted interface that any class can implement. As a result, the switch expression loses its exhaustiveness. Because the compiler can no longer track every possible subclass, it cannot guarantee that the switch will always yield a return value. This will trigger a compilation error unless you explicitly introduce a default case. However, by keeping the sealed keyword, the compiler recognizes the strict, fixed list of allowed subclasses, guaranteeing that all scenarios are handled without requiring a default branch.

Complete Triangle Case

```
case Triangle t ->
    String.format("Triangle base=%.2f height=%.2f area=%.2f",
        t.base(),
        t.height(),
        (t.base() * t.height()) / 2);
```

Section C

Practical Programming Task

C1: Project Setup

The screenshot displays the Visual Studio Code interface for a Spring Boot project. The Explorer view on the left shows the project structure, including the `src` directory with `main` and `test` subdirectories, and the `target` directory for compiled classes. The Output view in the center shows the Maven build process, including dependency resolution and compilation. The Terminal view on the right shows the execution of `mvnw.cmd spring-boot:run`, which starts the application. A chat window on the right provides instructions on how to run the application and troubleshoot port conflicts.

Terminal Output:

```
PS C:\Users\haile\Desktop\se4801-assignment1--ATE-1051-14> .\mvnw.cmd spring-boot:run
[INFO] Scanning for projects...
[INFO] Building shopwave-starter 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] >>> spring-boot:3.2.5:run (default-cli) > test-compile @ shopwave-starter >>
[INFO] --- resources:3.3.1:resources (default-resources) @ shopwave-starter ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO] --- compiler:3.11.0:compile (default-compile) @ shopwave-starter ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- resources:3.3.1:testResources (default-testResources) @ shopwave-starter ---
[INFO] skip non existing resourceDirectory C:\Users\haile\Desktop\se4801-assignment1--ATE-1051-14\src\test\resources
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ shopwave-starter ---
[INFO] Nothing to compile - all classes are up to date
[INFO] <<< spring-boot:3.2.5:run (default-cli) < test-compile @ shopwave-starter <<<
[INFO] --- spring-boot:3.2.5:run (default-cli) @ shopwave-starter ---
[INFO] Attaching agents: []
```

Chat Window:

Continue

What you got this error because I already had the application running in the background for you since our very first interaction!

When you tried to run `.\mvnw.cmd spring-boot:run`, your new terminal tried to grab Port 8080, but the server was already listening on that port from my background terminal.

I have just stopped the background process. **Port 8080 is now safely free.** You can switch back to your terminal right now and re-run:

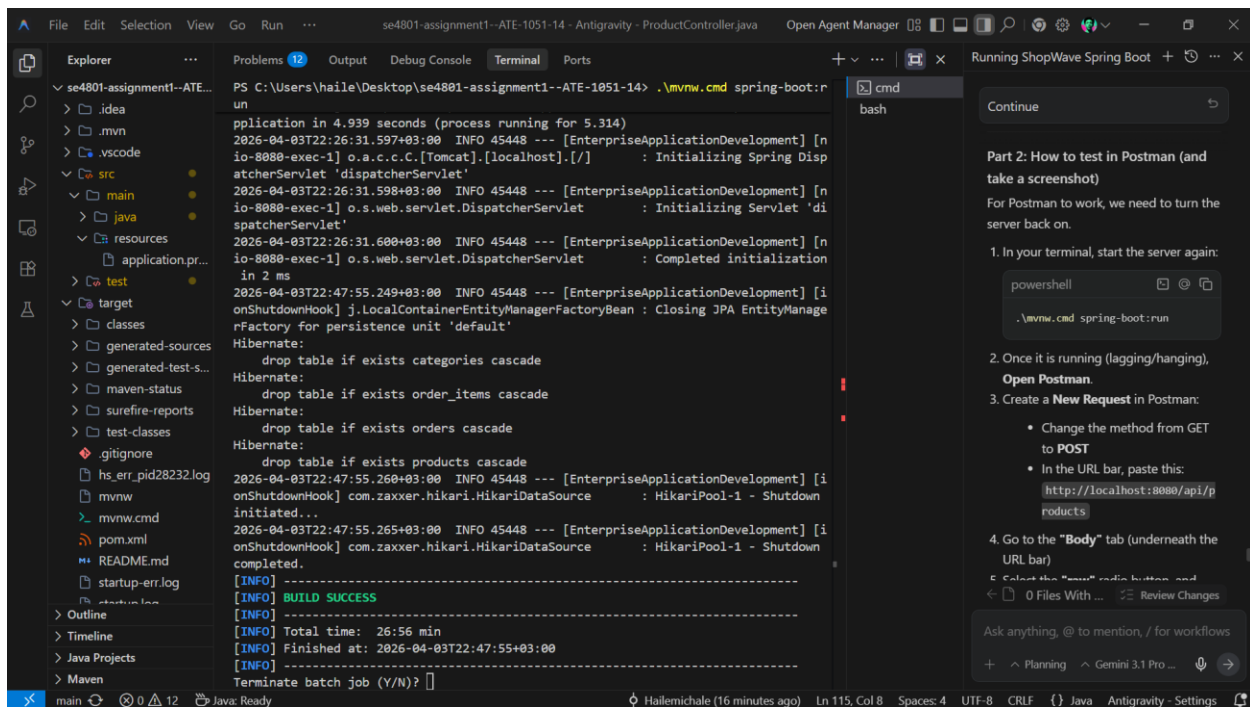
```
powershell
.\mvnw.cmd spring-boot:run
```

It should start up perfectly for you!

0 Files With ... Review Changes

Ask anything, @ to mention, / for workflows

+ Planning Gemini 3.1 Pro ...



C2: Domain Model

Implemented on my github repository

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/model>

C3: Repository and Service Layer

Implemented on github repository:

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/repository>

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/service>

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/model>

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/mapper>

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/exception>

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/dto>

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/controller>

C4: REST Controller

Implemented on github repository

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/controller>

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave/exception>

C5: Testing

Implemented on my github repostery

<https://github.com/Hailemichale/se4801-assignment1--ATE-1051-14-/tree/main/src/main/java/com/shopwave>

Sample API responses

The screenshot displays the Postman interface with a workspace named "HAILE HAILE's Workspace". The active tab is a POST request to `http://localhost:8080/api/products`. The request body is a JSON object:

```
1 {
2   "name": "iPhone 17 promax",
3   "description": "A brand new phone that stayes for limited time",
4   "price": 999.00,
5   "stock": 10
6 }
```

The response body is a JSON object:

```
1 {
2   "id": 2,
3   "name": "iPhone 17 promax",
4   "description": "A brand new phone that stayes for limited time",
5   "price": 999.00,
6   "stock": 10,
7   "categoryId": null,
8   "categoryName": null,
9   "createdAt": "2026-04-03T23:50:38.149998"
10 }
```

The status is **201 Created** with a response time of 17 ms and a size of 371 B. The interface also shows a sidebar with collections, environments, and specs, and a right-hand panel with a "Create Mechanical Ke..." tab and a "Send Request" button.

All tests passing

```

PS C:\Users\haile\Desktop\se4801-assignment1--ATE-1051-14> .\mvnw.cmd test
Hibernate:
    alter table if exists products
       add constraint FKog2rp4qthbtt2lfyhfo32lsw9
       foreign key (category_id)
       references categories
2026-04-03T22:09:05.207+03:00 INFO 31676 --- [EnterpriseApplicationDevelopment] [
    main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityMa
nagerFactory for persistence unit 'default'
2026-04-03T22:09:05.646+03:00 WARN 31676 --- [EnterpriseApplicationDevelopment] [
    main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view
is enabled by default. Therefore, database queries may be performed during view ren
dering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-04-03T22:09:06.400+03:00 INFO 31676 --- [EnterpriseApplicationDevelopment] [
    main] o.s.b.a.h2.H2ConsoleAutoConfiguration : H2 console available at
'/h2-console'. Database available at 'jdbc:h2:mem:shopwavedb'
2026-04-03T22:09:06.476+03:00 INFO 31676 --- [EnterpriseApplicationDevelopment] [
    main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 13 endpoint(s)
beneath base path '/actuator'
2026-04-03T22:09:06.545+03:00 INFO 31676 --- [EnterpriseApplicationDevelopment] [
    main] c.s.ShopWaveStarterApplicationTest : Started ShopWaveStarterA
pplicationTest in 2.334 seconds (process running for 17.704)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.373 s -- i
n com.shopwave.ShopWaveStarterApplicationTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 21.555 s
[INFO] Finished at: 2026-04-03T22:09:06+03:00
[INFO] -----
PS C:\Users\haile\Desktop\se4801-assignment1--ATE-1051-14> mvn

```